



Lecture 20: Two-Way Component Binding

INTERVIEW QUESTION

[› CORE](#)

How do you let a parent read and change a value that lives inside a child component?

Your `Input` component owns its value, but the parent needs to read and reset it. How do you set up two-way binding across the component boundary?

You already know how to pass data down to a child using normal props, and you know how to bind data to native HTML elements like `<input>` using `bind:value`. But what happens when you create your own custom components?

The Stagnant Parent Problem

Imagine you are building a custom `<PinCode>` component for a security application. Inside the component, you have the logic to handle four separate number inputs and concatenate them into a single `pin` string.

You import this `<PinCode>` into your `Login.svelte` parent component. The parent has a "Submit" button, but it cannot see the `pin` string because it is trapped inside the child's local state. Even worse, if the user enters the wrong PIN, the parent needs a way to wipe the child's input clean. If data strictly flows downwards, the parent is completely blind to the child's internal state and powerless to reset it.

The `$bindable()` Rune

In Svelte 5, you can explicitly mark a child component's prop as open for two-way communication using the `$bindable` rune.

By default, props are read-only. If the parent passes a value down, the child cannot alter it. But when the child wraps a prop in `$bindable()`, it declares: *"I am allowed to change this value, and if I do, the parent's variable will instantly update to match."*



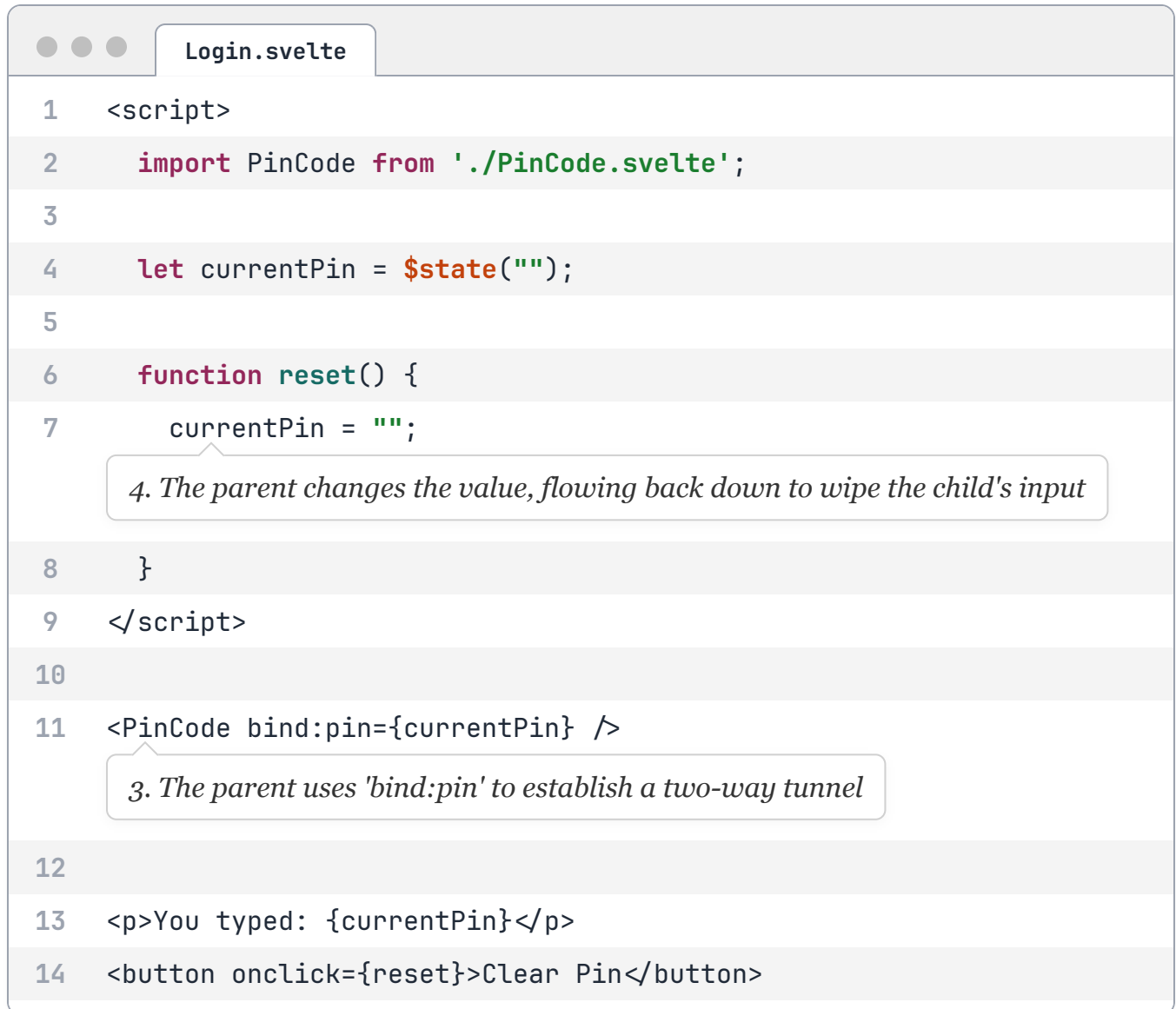
```
1  <script>
2    let { pin = $bindable() } = $props();
3  </script>
4
5  <input type="text" bind:value={pin} />
```

1. The child explicitly marks 'pin' as open for two-way binding

2. The child binds its local HTML input to the bindable prop

Binding Across the Boundary

Now that the child has given permission, the parent component can use the `bind:` directive directly on the custom component tag, exactly as it would on a native HTML `<input>` .



```
1  <script>
2    import PinCode from './PinCode.svelte';
3
4    let currentPin = $state("");
5
6    function reset() {
7      currentPin = "";
8    }
9  </script>
10
11  <PinCode bind:pin={currentPin} />
12
13  <p>You typed: {currentPin}</p>
14  <button onclick={reset}>Clear Pin</button>
```

4. The parent changes the value, flowing back down to wipe the child's input

3. The parent uses 'bind:pin' to establish a two-way tunnel

When the user types inside the child component, the native `<input>` updates the `pin` prop. Because `pin` is `$bindable()` , the change travels up the tunnel and updates the parent's `currentPin` state. When the parent clicks the reset button, `currentPin` changes to an empty string, traveling back down the tunnel to clear the child's `<input>` .

KEY TAKEAWAY

You can provide a default fallback value to a bindable prop just like any other prop: `let { pin = $bindable("0000") } = $props();`

Summary

To allow data to flow seamlessly in both directions across component boundaries, use the `$bindable()` rune. The child component must explicitly opt-in by wrapping its prop declaration in `$bindable()` . Once authorized, the parent component can use the `bind:propName` directive on the custom component. This creates a two-way synchronization tunnel, allowing both the parent and the child to read and modify the shared data safely.

	NORMAL PROPS	BINDABLE PROPS
Declaration	<pre>let { name } = \$props();</pre>	<pre>let { name = \$bindable() } = \$props();</pre>
Data Flow	Parent to Child (One-Way)	Two-Way Synchronization
Parent Usage	<pre><Child name={val} ></pre>	<pre><Child bind:name={val} ></pre>